MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Autonomous)

NEW PANVEL

PROJECT REPORT ON

"AI TRAVEL LUGGAGE PLANNER"

IN PARTIAL FULFILLMENT OF

MASTER OF **DATA ANALYTICS PART-II**

SEMESTER III – 2025-26

PROJECT GUIDE

PROF. SANJANA BHANGALE

SUBMITTED BY: **SHRUTI N YELVE**

ROLL NO: **4845**

Mahatma Education Society's

PILLAI COLLEGE OF ARTS,COMMERCE & SCIENCE

(Autonomous)

Re-accredited "A" Grade by NAAC (3rd Cycle)

Project Completion Certificate

THIS IS TO CERTIFY THAT

SHRUTI YELVE

of **M.Sc. Data Analytics Part - II** has completed the project titled **AI TRAVEL LUGGAGE PLANNER** of subject **ARTIFICAL INTELLIGENCE** under our guidance and supervision during the academic year 2025-26 in the department of Computer Science

PROJECT GUIDE **COURSE CORDINATOR** **HEAD OF DEPARTMENT**

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

INTRODUCTION

Travel planning is an exciting experience, but deciding *what to pack* often becomes confusing, time-consuming, and stressful. People usually worry about forgetting essentials, packing unnecessary items, or not preparing according to weather conditions and trip type. To solve this real-world problem, the **AI-Based Travel Luggage Planner** provides a smart, automated, and personalized packing solution.

This project uses **Python (Flask)** and **AI-style logic** to generate a customized luggage checklist based on the user's **destination**, **number of travel days**, **trip type**, and **live weather conditions**. By integrating the **OpenWeatherMap API**, the system automatically checks real-time weather and adjusts the packing list—such as adding jackets during cold weather or sunscreen and swimwear for beach locations.

The web interface is designed to be visually appealing, responsive, and user-friendly. It features **animated icons**, **attractive backgrounds**, and **intuitive checkboxes** for selecting items. Additionally, the application offers advanced functionalities such as **voice-based destination input**, **start/stop voice reading of the packing list**, and the ability to **download a PDF containing only the items selected by the user**. These enhancements make the system interactive and highly practical for modern travelers.

Overall, this AI-powered luggage planner transforms traditional packing methods by making them smarter, faster, and more accurate. It is an ideal solution for travelers who want a stress-free planning experience and a well-organized packing list tailored to their unique journey.

CODE AND OUTPUT

App.py

```
import os

import math

from datetime import datetime

from io import BytesIO

import requests

from flask import Flask, render_template, request, send_file, url_for

from dotenv import load_dotenv

from reportlab.pdfgen import canvas

from reportlab.lib.pagesizes import A4

from reportlab.lib.units import mm

from reportlab.lib import colors

load_dotenv()

app = Flask(__name__)

ICON_MAP = {

    'T-shirt': 'tshirt.svg',

    'Shirt/Top': 'tshirt.svg',

    'Dress': 'dress.svg',

    'Jeans/Trousers': 'pants.svg',

    'Shorts': 'shorts.svg',

    'Underwear': 'underwear.svg',

    'Socks': 'socks.svg',

    'Sleepwear': 'sleep.svg',

    'Jacket/Coat': 'jacket.svg',

    'Sweater/Hoodie': 'hoodie.svg',

    'Raincoat/Poncho': 'raincoat.svg',

    'Scarf': 'scarf.svg',

    'Gloves': 'gloves.svg',

    'Cap/Hat': 'hat.svg',

    'Swimwear': 'swim.svg',

    'Flip-flops': 'flipflops.svg',
```

'Walking Shoes': 'shoes.svg',

'Dress Shoes/Heels': 'heels.svg',

'Boots': 'boots.svg',

'Sunscreen (SPF 30+)': 'sunscreen.svg',

'Sunglasses': 'sunglasses.svg',

'Umbrella': 'umbrella.svg',

'Toothbrush': 'toothbrush.svg',

'Toothpaste': 'toothpaste.svg',

'Shampoo/Conditioner': 'bottle.svg',

'Soap/Body Wash': 'soap.svg',

'Deodorant': 'deodorant.svg',

'Razor/Trimmer': 'razor.svg',

'Makeup/Remover': 'makeup.svg',

'Skincare': 'skincare.svg',

'Medications': 'medicine.svg',

'First-aid (Band-aids etc.)': 'firstaid.svg',

'Sanitizer/Wipes': 'wipes.svg',

'Tissues': 'tissues.svg',

'Passport': 'passport.svg',

'Government ID': 'id.svg',

'Travel Tickets': 'ticket.svg',

'Visa/Itinerary': 'visa.svg',

'Cash & Cards': 'wallet.svg',

'Travel Insurance': 'insurance.svg',

'Power Bank': 'powerbank.svg',

'Phone + Charger': 'phone.svg',

'Laptop/Tablet + Charger': 'laptop.svg',

'Camera + Batteries': 'camera.svg',

'Universal Adapter': 'adapter.svg',

'Cables/USB': 'cable.svg',

'Backpack/Daypack': 'backpack.svg',

'Luggage Locks': 'lock.svg',

'Packing Cubes': 'cube.svg',

```
    'Reusable Bottle': 'bottle.svg',
    'Neck Pillow': 'pillow.svg',
    'Eye Mask/Earplugs': 'sleep.svg',
    'Jewelry (minimal)': 'jewelry.svg',
    'Belt': 'belt.svg',
    'Towel/Quick-dry': 'towel.svg',
    'Snacks': 'snack.svg',
}
TRIP_TYPES = ['leisure', 'business', 'beach', 'hiking', 'wedding', 'international']
def fetch_weather(location: str):
    """Fetch current weather for a location using OpenWeatherMap API.
    Returns (data, error)."""
    api_key = os.getenv('OPENWEATHER_API_KEY')
    if not api_key:
        return None, 'Missing OPENWEATHER_API_KEY. Set it in your environment or .env file.'
    try:
        resp = requests.get(
            'https://api.openweathermap.org/data/2.5/weather',
            params={'q': location, 'appid': api_key, 'units': 'metric'}, timeout=8
        )
        if resp.status_code != 200:
            return None, f'Weather API error: {resp.status_code} {resp.text[:120]}'
        j = resp.json()
        weather = {
            'name': j.get('name') or location,
            'country': (j.get('sys') or {}).get('country'),
            'temp': (j.get('main') or {}).get('temp'),
            'feels_like': (j.get('main') or {}).get('feels_like'),
            'humidity': (j.get('main') or {}).get('humidity'),
            'wind_speed': (j.get('wind') or {}).get('speed'),
            'condition': (j.get('weather') or [{}])[0].get('main'),
            'description': (j.get('weather') or [{}])[0].get('description'),
            'icon': (j.get('weather') or [{}])[0].get('icon'),
```

```
        }
        return weather, None
    except Exception as e:
        return None, f'Weather fetch failed: {e}'
def base_essentials(days: int):
    # quantities scaled by days
    tops = max(2, math.ceil(days * 0.7))
    bottoms = max(1, math.ceil(days / 3))
    underwear = days + 2
    socks = max(2, days)
    return {
        'Clothing': [
            ('T-shirt', tops),
            ('Shirt/Top', max(0, tops - 1)),
            ('Jeans/Trousers', bottoms),
            ('Shorts', 1 if days >= 2 else 0),
            ('Underwear', underwear),
            ('Socks', socks),
            ('Sleepwear', 1),
        ],
        'Footwear & Accessories': [
            ('Walking Shoes', 1),
            ('Belt', 1),
            ('Cap/Hat', 1),
            ('Sunglasses', 1),
        ],
        'Toiletries': [
            ('Toothbrush', 1),
            ('Toothpaste', 1),
            ('Shampoo/Conditioner', 1),
            ('Soap/Body Wash', 1),
            ('Deodorant', 1),
            ('Razor/Trimmer', 1),
```

```
        ('Skincare', 1),
        ('Makeup/Remover', 1),
        ('Sanitizer/Wipes', 1),
        ('Tissues', 1),
    ],
    'Health & Safety': [
        ('Medications', 1),
        ('First-aid (Band-aids etc.)', 1),
        ('Travel Insurance', 1),
    ],
    'Documents & Money': [
        ('Government ID', 1),
        ('Travel Tickets', 1),
        ('Cash & Cards', 1),
    ],
    'Electronics': [
        ('Phone + Charger', 1),
        ('Power Bank', 1),
        ('Cables/USB', 1),
    ],
    'Bags & Misc': [
        ('Backpack/Daypack', 1),
        ('Packing Cubes', 1),
        ('Luggage Locks', 1),
        ('Reusable Bottle', 1),
        ('Towel/Quick-dry', 1),
        ('Snacks', 1),
    ]
}
def weather_adjustments(weather: dict, trip_type: str):
    additions = {k: [] for k in ['Clothing', 'Footwear & Accessories', 'Toiletries']}
    if not weather:
        # If weather unavailable, suggest general items
```

```
        additions['Clothing'] += [('Jacket/Coat', 1)]
        additions['Footwear & Accessories'] += [('Umbrella', 1), ('Sunscreen (SPF 30+)', 1)]
        return additions

    temp = weather.get('temp')
    cond = (weather.get('condition') or '').lower()
    desc = (weather.get('description') or '').lower()

    is_rain = 'rain' in cond or 'rain' in desc or 'drizzle' in desc
    is_snow = 'snow' in cond or 'snow' in desc
    is_cold = temp is not None and temp < 15
    is_hot = temp is not None and temp > 30

    if is_cold:
        additions['Clothing'] += [('Jacket/Coat', 1), ('Sweater/Hoodie', 1), ('Scarf', 1), ('Gloves', 1)]
        additions['Footwear & Accessories'] += [('Boots', 1)]
    if is_hot or trip_type in ['beach']:
        additions['Clothing'] += [('Swimwear', 1)]
        additions['Footwear & Accessories'] += [('Flip-flops', 1), ('Cap/Hat', 1), ('Sunglasses', 1)]
        additions['Toiletries'] += [('Sunscreen (SPF 30+)', 1)]
    if is_rain:
        additions['Clothing'] += [('Raincoat/Poncho', 1)]
        additions['Footwear & Accessories'] += [('Umbrella', 1)]
    if is_snow:
        additions['Clothing'] += [('Jacket/Coat', 1), ('Sweater/Hoodie', 1), ('Scarf', 1), ('Gloves', 1)]
        additions['Footwear & Accessories'] += [('Boots', 1)]
    return additions
def trip_type_addons(trip_type: str):
    addons = {k: [] for k in ['Clothing', 'Footwear & Accessories', 'Documents & Money', 'Electronics', 'Bags & Misc']}
    t = (trip_type or '').lower()
    if t == 'business':
        addons['Clothing'] += [('Shirt/Top', 1), ('Dress', 1)]
```

```
        addons['Footwear & Accessories'] += [('Dress Shoes/Heels', 1)]
        addons['Electronics'] += [('Laptop/Tablet + Charger', 1)]
    if t == 'beach':
        addons['Clothing'] += [('Swimwear', 1), ('Shorts', 1)]
        addons['Footwear & Accessories'] += [('Flip-flops', 1), ('Cap/Hat', 1)]
    if t == 'hiking':
        addons['Footwear & Accessories'] += [('Boots', 1), ('Cap/Hat', 1)]
        addons['Bags & Misc'] += [('Reusable Bottle', 1)]
    if t == 'wedding':
        addons['Clothing'] += [('Dress', 1)]
        addons['Footwear & Accessories'] += [('Jewelry (minimal)', 1), ('Dress Shoes/Heels', 1)]
    if t == 'international':
        addons['Documents & Money'] += [('Passport', 1), ('Visa/Itinerary', 1)]
        addons['Electronics'] += [('Universal Adapter', 1)]
    # leisure is covered by base
    return addons

def merge_categories(cat_a: dict, cat_b: dict):
    out = {}
    keys = set(cat_a.keys()) | set(cat_b.keys())
    for k in keys:
        out[k] = []
        items = {}
        for name, qty in cat_a.get(k, []) + cat_b.get(k, []):
            items[name] = items.get(name, 0) + (qty or 0)
        out[k] = [(n, q) for n, q in items.items() if q > 0]
    return out

def build_packing_list(location: str, days: int, trip_type: str, weather: dict):
    base = base_essentials(days)
    weather_add = weather_adjustments(weather, trip_type)
    trip_add = trip_type_addons(trip_type)
```

```
    # Merge progressively
    merged = merge_categories(base, weather_add)
    merged = merge_categories(merged, trip_add)

    # Sort items alphabetically in each category for nice UI
    for k in list(merged.keys()):
        merged[k] = sorted(merged[k], key=lambda x: x[0].lower())
    return merged
@app.route('/')
def index():
    return render_template('index.html', trip_types=TRIP_TYPES)
@app.route('/planner', methods=['POST'])
def planner():
    location = request.form.get('location', '').strip()
    days = int(request.form.get('days', '4') or 4)
    trip_type = request.form.get('trip_type', 'leisure')
    weather, werr = fetch_weather(location) if location else (None, None)
    items = build_packing_list(location, days, trip_type, weather)

    return render_template(
        'planner.html',
        location=location or 'Your Trip',
        days=days,
        trip_type=trip_type,
        weather=weather,
        weather_error=werr,
        items=items,
        ICON_MAP=ICON_MAP
    )
@app.route('/pdf', methods=['GET', 'POST'])
def pdf():
    from reportlab.pdfgen import canvas
    from reportlab.lib.pagesizes import A4
```

```
from io import BytesIO
location = request.values.get("location", "Trip")
days = int(request.values.get("days", 4))
trip_type = request.values.get("trip_type", "leisure")

# Get selected items
selected_json = request.values.get("selected_items")
if selected_json:
    import json
    selected_list = json.loads(selected_json)
else:
    selected_list = None

# If selected items exist → only include them
if selected_list and len(selected_list) > 0:
    items = {
        "Selected Items": [(i["name"], i["qty"]) for i in selected_list]
    }
    weather = None  # Skip weather for selected-only mode
else:
    # Fallback: generate full packing list
    weather, _ = fetch_weather(location)
    items = build_packing_list(location, days, trip_type, weather)

buf = BytesIO()
c = canvas.Canvas(buf, pagesize=A4)
width, height = A4
c.setFont("Helvetica-Bold", 20)
c.drawString(40, height - 40, "AI Travel Luggage Planner")
c.setFont("Helvetica", 12)
c.drawString(40, height - 60,
             f"Location: {location} | Days: {days} | Trip: {trip_type.title()}")
y = height - 100
```

```
    # Draw items
    for cat, lst in items.items():
        c.setFont("Helvetica-Bold", 14)
        c.drawString(40, y, cat)
        y -= 20
        c.setFont("Helvetica", 11)
        for name, qty in lst:
            c.rect(40, y - 4, 10, 10)
            c.drawString(60, y, f"{name}  x{qty}")
            y -= 20
            if y < 60:
                c.showPage()
                y = height - 80
    c.save()
    buf.seek(0)
    return send_file(
        buf,
        as_attachment=True,
        download_name="selected_items.pdf",
        mimetype="application/pdf"
    )
```

Index.html

```
{% extends 'layout.html' %}
{% block body %}
<section class="hero">
  <div class="overlay"></div>
  <img class="bg" src="{{ url_for('static', filename='images/bg.svg') }}" alt="Travel Background" />
  <div class="hero-content">
    <h1 class="title">AI Travel Luggage Planner</h1>
    <p class="subtitle">Smart, weather-aware packing lists for stress-free trips.</p>
    <form class="planner-form" action="{{ url_for('planner') }}" method="POST">
      <div class="input-row">
```

```
      <div class="field">
        <label for="location">Destination</label>
        <div class="voice-wrap">
          <input id="location" name="location" placeholder="e.g., Goa, Manali, Mumbai, Paris" required />
          <button type="button" id="micBtn" class="icon-btn" title="Speak destination">
            <img src="{{ url_for('static', filename='icons/mic.svg') }}" alt="mic" />
          </button>
        </div>
      </div>
      <div class="field">
        <label for="days">Days</label>
        <input id="days" name="days" type="number" min="1" value="5" required />
      </div>
      <div class="field">
        <label for="trip_type">Trip type</label>
        <select id="trip_type" name="trip_type">
          {% for t in trip_types %}
          <option value="{{ t }}">{{ t.title() }}</option>
          {% endfor %}
        </select>
      </div>
    </div>
    <button class="primary" type="submit">Generate Packing List</button>
  </form>
 </div>
</section>
<footer class="footer">Made with ❤ for smarter travel.</footer>
{% endblock %}
```

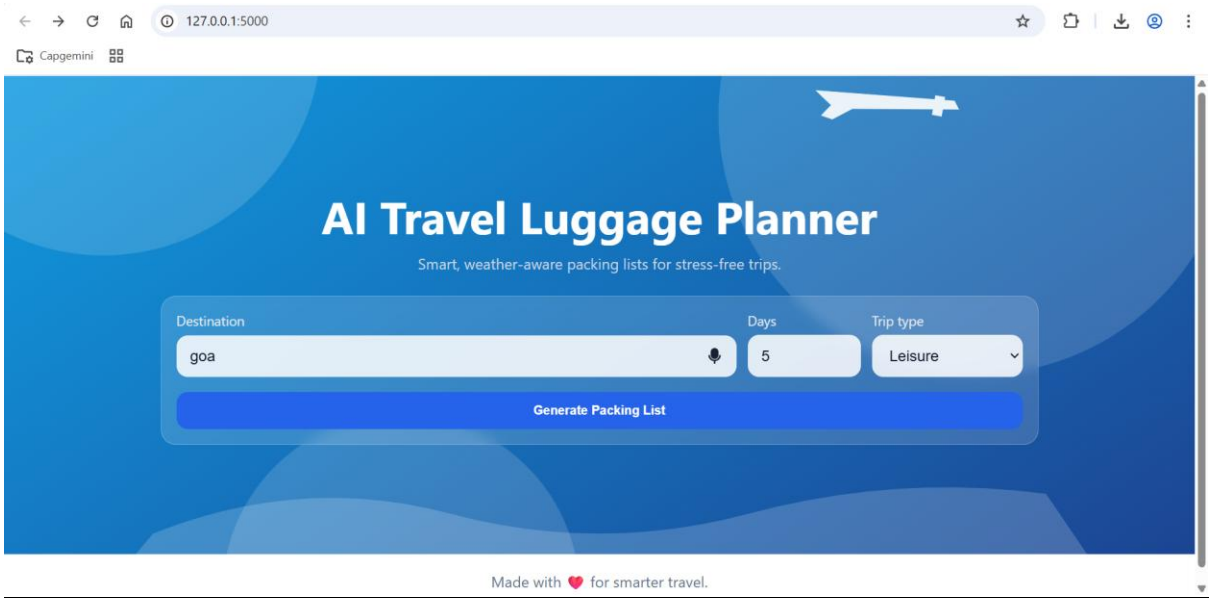
STYLE.CSS

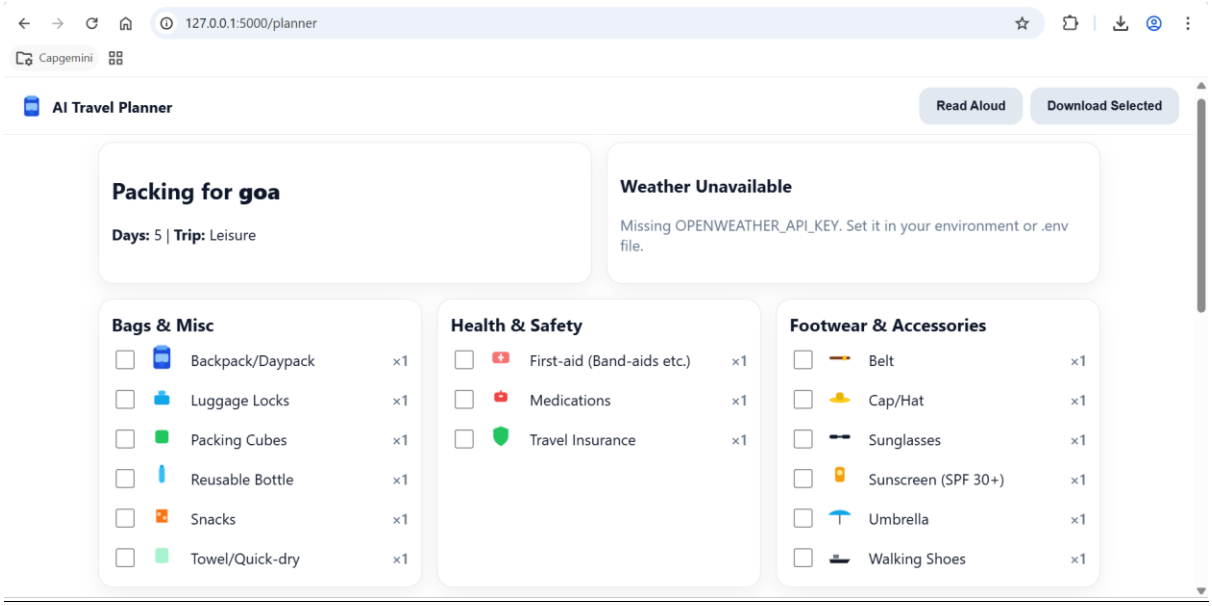
```
:root {
  --bg: #0f172a;
  --glass: rgba(255, 255, 255, 0.15);
  --card: #ffffff;
  --text: #0f172a;
  --muted: #64748b;
  --primary: #2563eb;
  --primary-600: #1d4ed8;
}
* { box-sizing: border-box; }
html, body { height: 100%; }
body { margin: 0; font-family: ui-sans-serif, system-ui, -apple-system, Segoe UI, Roboto, Arial, Noto Sans, "Helvetica Neue", "Apple Color Emoji", "Segoe UI Emoji"; color: var(--text); }
/* HERO */
.hero { position: relative; min-height: 92vh; display: grid; place-items: center; overflow: hidden; }
.hero .bg { position: absolute; inset: 0; width: 100%; height: 100%; object-fit: cover; }
.hero .overlay { position: absolute; inset: 0; background: radial-gradient(80% 60% at 50% 30%, rgba(2,6,23,0.4), rgba(2,6,23,0.85)); }
.hero .hero-content { position: relative; z-index: 2; padding: 24px; width: min(980px, 92vw); text-align: center; color: #fff; }
.title { font-size: clamp(2rem, 4vw, 3rem); margin: 0 0 8px; }
.subtitle { margin: 0 0 24px; color: #cbd5e1; }


.planner-form { display: grid; gap: 16px; backdrop-filter: blur(6px); background: var(--glass); border: 1px solid rgba(255,255,255,0.25); padding: 16px; border-radius: 16px; }
.input-row { display: grid; grid-template-columns: 1fr 120px 160px; gap: 12px; }
.field { display: grid; gap: 6px; text-align: left; }
.field label { color: #e2e8f0; font-size: 0.9rem; }
.field input, .field select { width: 100%; padding: 12px 14px; border-radius: 12px; border: 1px solid rgba(255,255,255,0.4); background: rgba(255,255,255,0.85); color: #0f172a; font-size: 1rem; }
.voice-wrap { position: relative; display: flex; align-items: center; }
.voice-wrap input { flex: 1; padding-right: 44px; }
```

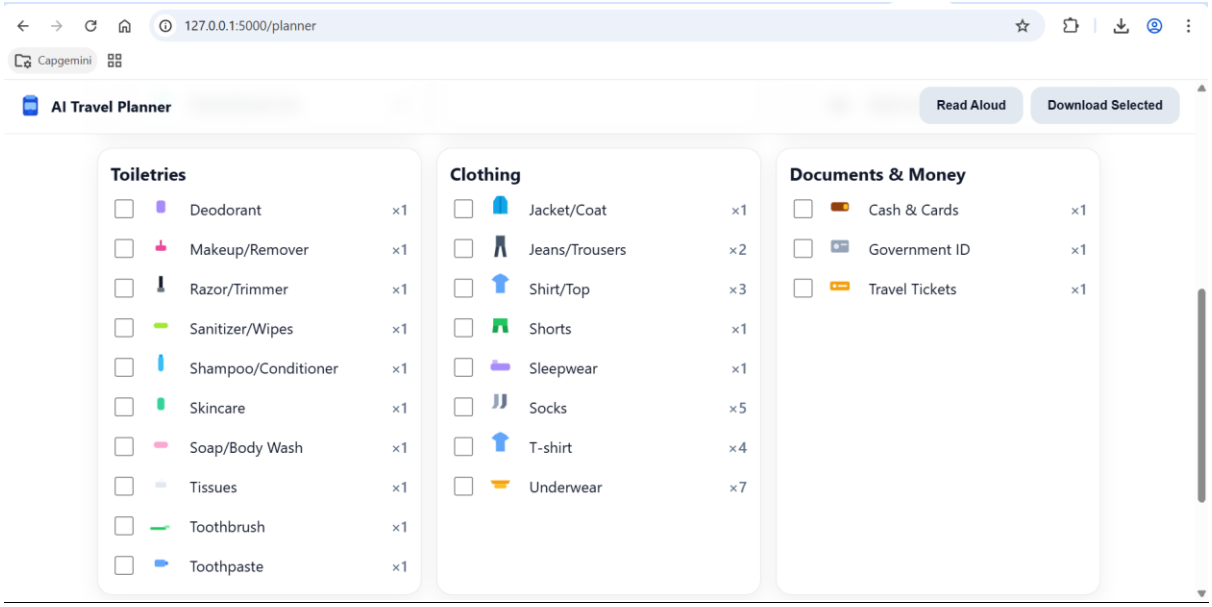
```
.icon-btn { position: absolute; right: 6px; top: 50%; transform: translateY(-50%); border: none; background: transparent; cursor: pointer; width: 34px; height: 34px; display: grid; place-items: center; border-radius: 8px; }

.icon-btn:hover { background: rgba(15, 23, 42, 0.1); }

.icon-btn img { width: 20px; height: 20px; filter: invert(0.05); }


.primary, .secondary { border: none; cursor: pointer; padding: 12px 18px; border-radius: 12px; font-weight: 600; }

.primary { background: var(--primary); color: #fff; }

.primary:hover { background: var(--primary-600); }

.secondary { background: #e2e8f0; color: #0f172a; }


.footer { padding: 18px; text-align: center; color: var(--muted); }


/* TOP */

.topbar { position: sticky; top: 0; display: flex; justify-content: space-between; align-items: center; gap: 12px; padding: 10px 16px; background: rgba(255,255,255,0.75); backdrop-filter: blur(8px); border-bottom: 1px solid #e5e7eb; z-index: 40; }

.brand { display: flex; align-items: center; gap: 8px; text-decoration: none; color: var(--text); font-weight: 700; }

.brand img { width: 28px; height: 28px; }

.actions { display: flex; gap: 8px; }


/* CONTENT */

.content { padding: 18px; max-width: 1100px; margin: 0 auto; }

.summary { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 16px; }

.card { background: var(--card); border: 1px solid #e5e7eb; border-radius: 16px; padding: 14px; box-shadow: 0 6px 20px rgba(2,6,23,0.06); }

.weather-row { display: flex; align-items: center; gap: 10px; }

.wicon { width: 64px; height: 64px; }

.muted { color: var(--muted); }


.grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 16px; }

.cat { margin: 0 0 8px; color: #0f172a; }

.items { list-style: none; margin: 0; padding: 0; display: grid; gap: 8px; }

.item label { display: grid; grid-template-columns: 28px 36px 1fr auto; align-items: center; gap: 10px; }
```

```
.item input[type=checkbox] { width: 20px; height: 20px; }

.icon { width: 30px; height: 30px; }

.qty { color: var(--muted); font-weight: 600; }


/* Animations */

@keyframes floatIn { from { transform: translateY(12px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }

.float-in { animation: floatIn 600ms ease both; }

.float-in.delay-1 { animation-delay: 120ms; }

@keyframes pulse { 0% { transform: scale(1); } 50% { transform: scale(1.08); } 100% { transform: scale(1); } }

.pulse { animation: pulse 2.6s ease-in-out infinite; }

@keyframes riseUp { from { transform: translateY(14px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }

.rise-up { animation: riseUp 700ms ease both; }

/* Mobile responsiveness */

@media (max-width: 900px) {

  .input-row { grid-template-columns: 1fr; }

  .summary { grid-template-columns: 1fr; }

  .grid { grid-template-columns: 1fr 1fr; }

}

@media (max-width: 560px) {

  .grid { grid-template-columns: 1fr; }

  .planner-form { padding: 12px; }

}
```

OUTPUT


127.0.0.1:5000
Capgemini
AI Travel Luggage Planner
Smart, weather-aware packing lists for stress-free trips.
Destination
goa
Days
5
Trip type
Leisure
Generate Packing List
Made with for smarter travel.


127.0.0.1:5000/planner
Capgemini
AI Travel Planner
Read Aloud
Download Selected
Packing for goa
Days: 5 | Trip: Leisure
Weather Unavailable
Missing OPENWEATHER_API_KEY. Set it in your environment or .env file.
Bags & Misc
Backpack/Daypack ×1
Luggage Locks ×1
Packing Cubes ×1
Reusable Bottle ×1
Snacks ×1
Towel/Quick-dry ×1
Health & Safety
First-aid (Band-aids etc.) ×1
Medications ×1
Travel Insurance ×1
Footwear & Accessories
Belt ×1
Cap/Hat ×1
Sunglasses ×1
Sunscreen (SPF 30+) ×1
Umbrella ×1
Walking Shoes ×1



CONCLUSION

The AI-Based Travel Luggage Planner successfully transforms the traditional packing process into an intelligent, automated, and user-friendly experience. By integrating Python, Flask, real-time weather APIs, and AI-logic, the system generates personalized packing lists based on destination, travel duration, trip type, and live climate conditions. The inclusion of features like voice input, responsive UI, animated icons, dynamic PDF generation, and the ability to download only selected items makes the solution both practical and modern.

This project eliminates guesswork, reduces human error, and ensures travelers are better prepared for any type of journey. It demonstrates how technology can simplify everyday tasks and enhance user convenience. Overall, the system meets its objectives effectively and proves to be a robust, scalable, and user-centric application.

FUTURE SCOPE

1. Integration of 5-Day or 7-Day Weather Forecast

Instead of using only current weather, the system can analyze extended forecasts and recommend day-wise packing suggestions for more accuracy.

2. User Login and Cloud Sync

Users can create accounts, save their trips, packing history, and download previous packing lists from anywhere.

3. AI/ML-Based Personal Packing Assistant

Using machine learning, the system can learn user preferences such as clothing style, travel habits, or frequently packed items, and auto-suggest customized lists.

4. Luggage Weight Estimation

The application can calculate an approximate luggage weight based on selected items and alert users about airline weight limits.

5. Multi-Language Support

Supporting regional and global languages (Hindi, Marathi, French, Spanish, etc.) would make the tool accessible to a wider audience.

6. Mobile App Conversion

The system can be converted into a cross-platform mobile app using Flutter or React Native for offline access and push notifications.